

OPENAI AGENTKIT & RESPONSES API

O'Reilly Live Training — Lucas Soares — Quick Reference

Responses API | Agents SDK | Agent Builder | ChatKit | Evals

1. THE THREE-LAYER STACK

1

Responses API

The Primitive
Direct API calls
Built-in tools & server-side state
Released March 2025

2

Agents SDK

The Framework
Multi-agent orchestration
Guardrails & tracing
Python + TypeScript, 100+ LLMs

3

AgentKit

The Product Platform
Agent Builder + ChatKit
Connectors + Evals
Launched DevDay Oct 2025

Core Principle

Each layer builds on the previous — pick the level of abstraction you need. Prototype in Agent Builder, export to SDK for production hardening. No vendor lock-in. Standard API pricing — no platform fee.

2. RESPONSES API — THE PRIMITIVE

Key Innovations

- **Items-based I/O** — polymorphic output: text, tool calls, reasoning
- **Server-side state** via `previous_response_id`
- **Reasoning preservation** across conversation turns
- Replaces Chat Completions API *and* Assistants API

Built-in Tools

- `web_search_preview` — live web search
- `file_search` — RAG over uploaded files
- `code_interpreter` — Python sandbox
- `computer_use_preview` — GUI automation
- `mcp` — Model Context Protocol servers

Basic Call

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-4o",
    input="Summarize the latest AI news",
    tools=[{"type": "web_search_preview"}]
)
print(response.output_text)
```

Multi-turn (Server-side State)

```
from openai import OpenAI
client = OpenAI()

r1 = client.responses.create(
    model="gpt-4o",
    input="What is the Responses API?"
)
# Continue — server manages history:
r2 = client.responses.create(
    model="gpt-4o",
    input="Give me a code example",
    previous_response_id=r1.id
)
```

3. AGENTS SDK — THE FRAMEWORK

5 Core Primitives `pip install openai-agents`

- **Agents** — LLM + tools + instructions
- **Runner** — automated agent loop execution
- **Handoffs** — multi-agent delegation
- **Guardrails** — parallel input/output validation
- **Tracing** — built-in observability (spans, traces)

Use SDK When...

- Custom server-side state management
- Human-in-the-loop approval flows
- Fine-grained tool execution control
- Multi-runtime or non-OpenAI deployment
- Full code ownership required

Basic Agent + Runner

```
from agents import Agent, Runner

agent = Agent(
    name="Research Assistant",
    instructions="You are a research assistant.",
    model="gpt-4o",
    tools=[web_search]
)

result = await Runner.run(
    agent, "Summarize recent AI papers"
)
print(result.final_output)
```

Multi-agent Handoffs

```
from agents import Agent, Runner

billing = Agent(name="Billing Agent", ...)
technical = Agent(name="Tech Agent", ...)
general = Agent(name="General Agent", ...)

triage = Agent(
    name="Triage",
    instructions="Route to the right agent.",
    handoffs=[billing, technical, general]
)

result = await Runner.run(
    triage, "I can't access my invoice"
)
```

4. AGENTKIT PLATFORM — THE PRODUCT LAYER

Agent Builder

Visual drag-and-drop canvas for composing multi-step agent workflows — no orchestration glue code. Versioned, publishable workflows with per-node traces and interactive preview testing.

Connector Registry

Pre-built data connectors: Dropbox, Google Drive, SharePoint, and any MCP server. Add as a node in any workflow — no custom integration code needed.

ChatKit

Embeddable React component + Web Component chat UI. Handles auth, streaming, file upload, tool invocation display, chain-of-thought visualization, and suggestion chips.

Evals for Agents

Trace grading, dataset building from live runs, and automated prompt optimization. Closes the Build → Test → Deploy → Measure → Improve cycle.

5. AGENT BUILDER — NODE TYPES

Start Entry point — accepts initial user input	Agent Calls an LLM with instructions + tools	Classifier Routes to one of N branches by output category
If / Else Conditional logic on data values	File Search Vector search over uploaded documents (RAG)	Transform Maps data between incompatible node outputs/ inputs
MCP Calls a hosted MCP server tool (3rd-party data/ actions)	Guardrails Input/output safety validation before/after agents	End Returns final output to the caller

Guardrails Protect Against • PII detection — Block or Redact mode • Jailbreak attempts • Prompt injection attacks • Off-topic / out-of-scope requests • Sensitive data leakage in responses	Versioning & Deployment • Each version: stable ID + immutable graph snapshot • Safe rollbacks — prior versions stay accessible • Staged promotion: dev → staging → prod • ChatKit — embed in minutes, point at workflow ID • SDK Export — Python/TS for custom server control
--	--

6. CHATKIT — EMBEDDABLE CHAT UI

React Integration <pre>import { ChatKit, useChatKit } from '@openai/chatkit-react'; const chatkit = useChatKit({ api: { getClientSecret } }); return <ChatKit control={chatkit.control} />;</pre>	Backend Token Exchange <pre>// POST /api/create-session const response = await fetch("https://api.openai.com/v1/chatkit/sessions", { method: "POST", headers: { Authorization: `Bearer \${API_KEY}` }, body: JSON.stringify({ workflow_id: WF_ID }) }); const { client_secret } = await response.json(); // Return client_secret — never the API key</pre>
--	---

7. EVALS LOOP & LAYER DECISION GUIDE

The Evals Cycle 1 Build 2 Test 3 Deploy 4 Measure 5 Improve • Trace Grading — grade real conversation outputs • Datasets — build eval sets from live runs • Prompt Optimization — iterate instructions using eval scores	When to Use Each Layer <table><tr><th>What you need</th><th>Use</th></tr><tr><td>Direct API control / learning</td><td>Responses API</td></tr><tr><td>Multi-agent orchestration, custom state</td><td>Agents SDK</td></tr><tr><td>Ship a chat UI fast</td><td>ChatKit</td></tr><tr><td>Visual workflows, non-dev teams</td><td>Agent Builder</td></tr><tr><td>Measure & improve agent quality</td><td>Evals</td></tr><tr><td>Prototype → production hardening</td><td>Builder → SDK</td></tr></table>	What you need	Use	Direct API control / learning	Responses API	Multi-agent orchestration, custom state	Agents SDK	Ship a chat UI fast	ChatKit	Visual workflows, non-dev teams	Agent Builder	Measure & improve agent quality	Evals	Prototype → production hardening	Builder → SDK
What you need	Use														
Direct API control / learning	Responses API														
Multi-agent orchestration, custom state	Agents SDK														
Ship a chat UI fast	ChatKit														
Visual workflows, non-dev teams	Agent Builder														
Measure & improve agent quality	Evals														
Prototype → production hardening	Builder → SDK														

8. DEMO WORKFLOWS (AGENT BUILDER)

Demo 1 — Course Assistant

Start → Agent → End

- Minimal single-node workflow
- Set system instructions on Agent node
- Per-node trace inspection in preview
- Best starting template for any assistant
- *Model: gpt-4o-mini, chat history on*

Demo 3 — File Search RAG (SEC 10-K)

Query Agent → File Search → Transform → Evidence Summarizer

- 10 SEC 10-K filings in vector store
- Transform node bridges typed output mismatch
- Evidence Summarizer with cited sources
- *"What does NVIDIA say about AI data centers?"*

Demo 2 — Support Triage

Guardrails → Classifier → Billing / Technical / General or Safety Refusal

- PII detection (Block mode)
- Jailbreak detection enabled
- Confidence-based routing
- Safety Refusal for policy violations

ChatKit App (Module 4)

Next.js starter embedding a published Agent Builder workflow via ChatKit.

- Auth via short-lived client secrets
- Streaming responses out of the box
- File upload support built-in
- Run: `npm run dev` (requires Node.js 18+)

9. NOTEBOOKS & SETUP

Core Learning Notebooks

1. **Intro to Agents** — Responses API fundamentals
2. **Simple Chat** — conversational interfaces
3. **Structured Outputs** — guaranteed JSON schemas
4. **Agentic Workflows** — complex agent pipelines
5. **File Search & RAG** — vector store retrieval
6. **Video Script + Images** — multi-modal vision agents
7. **Chat with Paper Agent** — document Q&A
8. **Research Report Generation** — end-to-end research agent

Setup (uv)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
make all # venv + deps + kernel
source .venv/bin/activate
# .env:
OPENAI_API_KEY=sk-...
```

Key Links

- Agents SDK: github.com/openai/openai-agents-python
- Agent Builder Guide: platform.openai.com/docs/guides/agent-builder
- ChatKit Guide: platform.openai.com/docs/guides/chatkit
- OpenAI Cookbook: cookbook.openai.com

KEY TAKEAWAYS

- **Start with Responses API** — maximum control & deepest learning; replaces Chat Completions API + Assistants API (March 2025)
- **Add Agents SDK** when you need multi-agent orchestration, handoffs, guardrails, and built-in tracing
- **Use Agent Builder** for visual workflow composition — great for non-dev teams, rapid prototyping, and Evals in the UI
- **Deploy via ChatKit** when shipping to real users — auth, streaming, and file upload are handled for you
- **Each layer builds on the previous** — prototype in Agent Builder, export to SDK for production; no vendor lock-in at SDK level